

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: data = pd.read_csv(r"zoo.csv")
data.head()
```

```
Out[2]: animal_name  hair  feathers  eggs  milk  airborne  aquatic  predator  toothed  backbone
```

|   |          |   |   |   |   |   |   |   |   |
|---|----------|---|---|---|---|---|---|---|---|
| 0 | aardvark | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 1 | antelope | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 2 | bass     | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
| 3 | bear     | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 4 | boar     | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |



```
In [3]: data.shape
```

```
Out[3]: (101, 18)
```

```
In [4]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 18 columns):
#   Column          Non-Null Count  Dtype
---  -
0   animal_name     101 non-null   str
1   hair            101 non-null   int64
2   feathers        101 non-null   int64
3   eggs            101 non-null   int64
4   milk            101 non-null   int64
5   airborne        101 non-null   int64
6   aquatic         101 non-null   int64
7   predator        101 non-null   int64
8   toothed         101 non-null   int64
9   backbone        101 non-null   int64
10  breathes        101 non-null   int64
11  venomous        101 non-null   int64
12  fins            101 non-null   int64
13  legs            101 non-null   int64
14  tail            101 non-null   int64
15  domestic        101 non-null   int64
16  catsize         101 non-null   int64
17  class_type      101 non-null   int64
dtypes: int64(17), str(1)
memory usage: 14.9 KB
```

```
In [5]: data.isna().sum()
```

```
Out[5]: animal_name    0
        hair          0
        feathers      0
        eggs          0
        milk          0
        airborne      0
        aquatic        0
        predator       0
        toothed        0
        backbone       0
        breathes       0
        venomous       0
        fins           0
        legs           0
        tail           0
        domestic       0
        catsize        0
        class_type     0
        dtype: int64
```

```
In [6]: data.columns
```

```
Out[6]: Index(['animal_name', 'hair', 'feathers', 'eggs', 'milk', 'airborne',
              'aquatic', 'predator', 'toothed', 'backbone', 'breathes', 'venomous',
              'fins', 'legs', 'tail', 'domestic', 'catsize', 'class_type'],
             dtype='str')
```

```
In [7]: data['class_type'].unique
```

```
Out[7]: <bound method Series.unique of 0      1
        1      1
        2      4
        3      1
        4      1
        ..
        96     1
        97     6
        98     1
        99     7
       100     2
        Name: class_type, Length: 101, dtype: int64>
```

```
In [9]: data.class_type.value_counts()
```

```
Out[9]: class_type
        1      41
        2      20
        4      13
        7      10
        6       8
        3       5
        5       4
        Name: count, dtype: int64
```

```
In [10]: data.duplicated().sum()
```

Out[10]: np.int64(0)

In [11]: data.animal\_name.value\_counts()

Out[11]: animal\_name  
 frog 2  
 aardvark 1  
 antelope 1  
 bass 1  
 bear 1  
 ..  
 wallaby 1  
 wasp 1  
 wolf 1  
 worm 1  
 wren 1  
 Name: count, Length: 100, dtype: int64

In [12]: from sklearn.preprocessing import LabelEncoder

In [13]: lb = LabelEncoder()

In [14]: data['animal\_name'] = lb.fit\_transform(data.animal\_name)  
 data['class\_type'] = lb.fit\_transform(data.class\_type)

In [15]: data.head()

Out[15]:

|   | animal_name | hair | feathers | eggs | milk | airborne | aquatic | predator | toothed | backbone |
|---|-------------|------|----------|------|------|----------|---------|----------|---------|----------|
| 0 | 0           | 1    | 0        | 0    | 1    | 0        | 0       | 1        | 1       |          |
| 1 | 1           | 1    | 0        | 0    | 1    | 0        | 0       | 0        | 1       |          |
| 2 | 2           | 0    | 0        | 1    | 0    | 0        | 1       | 1        | 1       |          |
| 3 | 3           | 1    | 0        | 0    | 1    | 0        | 0       | 1        | 1       |          |
| 4 | 4           | 1    | 0        | 0    | 1    | 0        | 0       | 1        | 1       |          |

In [16]: data.info()

```

<class 'pandas.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 18 columns):
#   Column          Non-Null Count  Dtype
---  -
0   animal_name     101 non-null   int64
1   hair            101 non-null   int64
2   feathers        101 non-null   int64
3   eggs            101 non-null   int64
4   milk            101 non-null   int64
5   airborne        101 non-null   int64
6   aquatic         101 non-null   int64
7   predator        101 non-null   int64
8   toothed         101 non-null   int64
9   backbone        101 non-null   int64
10  breathes        101 non-null   int64
11  venomous        101 non-null   int64
12  fins            101 non-null   int64
13  legs            101 non-null   int64
14  tail            101 non-null   int64
15  domestic        101 non-null   int64
16  catsize         101 non-null   int64
17  class_type      101 non-null   int64
dtypes: int64(18)
memory usage: 14.3 KB

```

```
In [17]: data.animal_name.value_counts()
```

```

Out[17]: animal_name
25      2
0       1
1       1
2       1
3       1
..
95      1
96      1
97      1
98      1
99      1
Name: count, Length: 100, dtype: int64

```

```
In [18]: data.class_type.value_counts()
```

```

Out[18]: class_type
0      41
1      20
3      13
6      10
5       8
2       5
4       4
Name: count, dtype: int64

```

```

In [20]: # import classification algorithm
from sklearn.linear_model import LogisticRegression

```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier

```

```

In [21]: from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score

```

```

In [22]: # split data into X and y
        X = data.iloc[:, :-1]
        X.head(2)

```

```

Out[22]:
   animal_name  hair  feathers  eggs  milk  airborne  aquatic  predator  toothed  backbon
0            0    1      0      0    1          0         0          1         1
1            1    1      0      0    1          0         0          0         1

```

```

In [23]: y = data.iloc[:, -1]
        y.head(2)

```

```

Out[23]:
0    0
1    0
Name: class_type, dtype: int64

```

```

In [24]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2, random_state=42)

```

```

In [25]: models = {
        'LogisticRegression': LogisticRegression(),
        'DecisionTreeClassifier': DecisionTreeClassifier(),
        'RandomForestClassifier': RandomForestClassifier(),
        'KNeighborsClassifier': KNeighborsClassifier(),
        'SVC': SVC(),
        'XGBClassifier': XGBClassifier(),
    }

```

```

In [26]: for name, model in models.items():
        model.fit(X_train,y_train)
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test,y_pred)
        print(f"Model name: {name}")
        print(f"Accuracy score: {acc}")
        print("-----")

```

```
D:\my project\codeit_datascience\data_science\venv\Lib\site-packages\sklearn\linear_model\_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 100 iterations(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max\_iter=100).  
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

[https://scikit-learn.org/stable/modules/linear\\_model.html#logistic-regression](https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)  
n\_iter\_i = \_check\_optimize\_result(

Model name: LogisticRegression

Accuracy score: 0.9523809523809523

-----

Model name: DecisionTreeClassifier

Accuracy score: 0.9523809523809523

-----

Model name: RandomForestClassifier

Accuracy score: 0.9523809523809523

-----

Model name: KNeighborsClassifier

Accuracy score: 0.42857142857142855

-----

Model name: SVC

Accuracy score: 0.5714285714285714

-----

Model name: XGBClassifier

Accuracy score: 0.9523809523809523

-----

```
In [27]: # Lets scale the 'X' input parameters
from sklearn.preprocessing import MinMaxScaler
```

```
In [28]: X.head(2)
```

```
Out[28]:
```

|   | animal_name | hair | feathers | eggs | milk | airborne | aquatic | predator | toothed | backbone |
|---|-------------|------|----------|------|------|----------|---------|----------|---------|----------|
| 0 | 0           | 1    | 0        | 0    | 1    | 0        | 0       | 1        | 1       |          |
| 1 | 1           | 1    | 0        | 0    | 1    | 0        | 0       | 0        | 0       | 1        |



```
In [29]: scaler = MinMaxScaler()
```

```
In [31]: X_Scaled_data = scaler.fit_transform(X)
X_Scaled_data
```

```
Out[31]: array([[0.          , 1.          , 0.          , ..., 0.          , 0.          ,
                1.          ],
               [0.01010101, 1.          , 0.          , ..., 1.          , 0.          ,
                1.          ],
               [0.02020202, 0.          , 0.          , ..., 1.          , 0.          ,
                0.          ],
               ...,
               [0.97979798, 1.          , 0.          , ..., 1.          , 0.          ,
                1.          ],
               [0.98989899, 0.          , 0.          , ..., 0.          , 0.          ,
                0.          ],
               [1.          , 0.          , 1.          , ..., 1.          , 0.          ,
                0.          ]], shape=(101, 17))
```

```
In [32]: X_Scaled_data = pd.DataFrame(X_Scaled_data, columns = X.columns)
X_Scaled_data.head(2)
```

```
Out[32]:
```

|   | animal_name | hair | feathers | eggs | milk | airborne | aquatic | predator | toothed | backbo |
|---|-------------|------|----------|------|------|----------|---------|----------|---------|--------|
| 0 | 0.000000    | 1.0  | 0.0      | 0.0  | 1.0  | 0.0      | 0.0     | 1.0      | 1.0     |        |
| 1 | 0.010101    | 1.0  | 0.0      | 0.0  | 1.0  | 0.0      | 0.0     | 0.0      | 1.0     |        |

```
In [33]: # new 'X' and 'y'
X_train, X_test, y_train, y_test = train_test_split(X_Scaled_data, y, test_size=0.2, ra
```

```
In [34]: for name, model in models.items():
          model.fit(X_train, y_train)
          y_pred = model.predict(X_test)
          acc = accuracy_score(y_test, y_pred)
          print(f"Model name: {name}")
          print(f"Accuracy score: {acc}")
          print("-----")
```

```
Model name: LogisticRegression
Accuracy score: 0.9523809523809523
-----
Model name: DecisionTreeClassifier
Accuracy score: 0.9523809523809523
-----
Model name: RandomForestClassifier
Accuracy score: 0.9523809523809523
-----
Model name: KNeighborsClassifier
Accuracy score: 0.9523809523809523
-----
Model name: SVC
Accuracy score: 0.9523809523809523
-----
Model name: XGBClassifier
Accuracy score: 0.9523809523809523
-----
```

```
In [35]: # After scaling using 'minmaxscaler' all model predict accuracy is 95%
```

```
In [37]: from sklearn.metrics import confusion_matrix
```

```
In [38]: cm = confusion_matrix(y_test,y_pred)
cm
```

```
Out[38]: array([[12,  0,  0,  0,  0,  0],
                [ 0,  2,  0,  0,  0,  0],
                [ 0,  0,  0,  1,  0,  0],
                [ 0,  0,  0,  2,  0,  0],
                [ 0,  0,  0,  0,  3,  0],
                [ 0,  0,  0,  0,  0,  1]])
```

```
In [ ]:
```